PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

UE24CS341A — Software Engineering

Software Test Plan

for

Student Attendance Management System (SAMS)

Version 1.1

Based on SRS v1.1, Team 3, 30-10-2026

Date: 30-10-2026, updated to reflect the work done in Sprints 1 and 2

Prepared by Team 3

Name	SRN
Anmol Vyas	PES2UG24CS072
Anupam Rajesh Tiwari	PES2UG24CS073
Anupriya	PES2UG24CS074
Anusha G	PES2UG24CS075

Revision History

Version	Date	Author	Change Summary
0.1	13-09-2026	Anupam Rajesh Tiwari	Initial skeleton and Section 1 (Introduction); Sections 2-3; UC-01 and UC-02 test coverage.
0.2	14-09-2026	Anusha G	Sections 4-6.
0.3	14-09-2026	Anusha G	UC-03 and UC-07 test coverage.
0.4	15-09-2026	Anupriya	Sections 7-9.
0.5	15-09-2026	Anupriya	UC-04 and UC-05 test coverage.
0.6	16-09-2026	Anmol Vyas	Sections 10-12; restructured the document to the department's 15-section template.
0.7	16-09-2026	Anmol Vyas	UC-06 test coverage; cross-checked Section 13 against the SRS's RTM.
0.8	17-09-2026	Anupam Rajesh Tiwari	Section 14 (Test Metrics and Reporting).
1.0	17-09-2026	Team 3	Trimmed Section 13 to a compact traceability table matching the ATM sample template; reviewed together; finalised for submission.
1.1	30-10-2026	Anupriya	Updated to reflect the work done in Sprints 1 and 2: SAM-F-008, SAM-F-009 and SAM-F-014 added to Sections 3 and 13; Docker, GitHub Actions, SonarCloud, Bandit, coverage.py and ruff in Section 6; actual schedule, deliverables and metrics; open items in 13.1 marked resolved.

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Asha Kumari	asha.hebbar@gmail.com	Pending

Table of Contents

1. Introduction **4**
2. Test Items **5**
3. Features to be Tested **5**
4. Features Not to be Tested **5**
5. Test Approach / Strategy **7**
5.1 Security Validation 7
6. Test Environment **8**
7. Test Schedule **8**
8. Test Deliverables **8**
9. Roles and Responsibilities **10**
10. Risks and Mitigation **11**
11. Assumptions and Dependencies **11**
12. Suspension and Resumption Criteria **12**
13. Test Case Management and Traceability **12**
13.1 Open Items Raised While Validating Traceability 13
14. Test Metrics and Reporting **14**
15. Approvals **14**

1. Introduction

Purpose

This document is the Software Test Plan (STP) for the Student Attendance Management System (SAMS), prepared by Team 3 for the UE24CS341A Software Engineering mini-project. It defines objectives, scope, strategy, resources, schedule, and responsibilities for testing SAMS against the requirements and use cases defined in the Phase-1 SRS (v1.0).

This document is being finalised ahead of the Phase-2 Project Plan, on the course coordinator's instruction, since test design can proceed directly from the approved SRS and use-case model without waiting on the WBS/scheduling artifacts that Phase-2 covers.

Scope

Testing covers all seven use cases defined in Section 7 of the SRS (UC-01 through UC-07), which together implement the 20 functional requirements (SAM-F-001 to SAM-F-020) and 5 security requirements (SAM-SEC-001 to SAM-SEC-005). Excluded from this phase: the Django/MySQL framework internals themselves, and full-scale performance/load testing (see Sections 3 and 4).

References

- Software Requirements Specification, Student Attendance Management System, v1.0, Team 3, 03-09-2026 — in particular its Requirements Traceability Matrix (Section 8), which this plan builds on rather than repeats (see Section 13).
- Jackfruit Mini-Project Guidelines 2026, PES University Dept. of CSE.
- Department Software Test Plan template (ATM System sample) — this document follows its 15-section structure, including the lighter, example-based treatment of Section 13.
- Django Documentation, https://docs.djangoproject.com/ — framework-specific test tooling (e.g. Django's TestCase class).

Definitions

Term	Definition
SAMS	Student Attendance Management System
SRS	Software Requirements Specification
STP	Software Test Plan (this document)
RTM	Requirements Traceability Matrix
UT / IT / ST	Unit Test / Integration Test / System Test
SEC	Security-focused test case, verifying a SAM-SEC requirement specifically
UC	Use Case, as defined in SRS Section 7

2. Test Items

The following modules, as named in the SRS's Requirements Traceability Matrix (Section 8), are the items under test:

- Authentication Module (login, role-based access) — SAM-F-001 to SAM-F-003
- Student Module (add/update/search student records, view own history) — SAM-F-004 to SAM-F-006, SAM-F-016
- Faculty Module (faculty records, assigned subjects) — SAM-F-007
- Subject Module (subject creation, faculty assignment) — SAM-F-008, SAM-F-009
- Attendance Module (marking, modification, duplicate prevention) — SAM-F-010 to SAM-F-014
- Calculation Module (attendance percentage) — SAM-F-015
- Notification Module (low-attendance alerts) — SAM-F-017
- Report Module (student-wise, subject-wise, class-wise reports; download/print) — SAM-F-018 to SAM-F-020
- Access Control Module (role enforcement, data isolation) — SAM-SEC-001, SAM-SEC-002, SAM-SEC-004
- Audit Module (logging of administrative actions) — SAM-SEC-005

3. Features to be Tested

Features are mapped to SRS requirement IDs, grouped by the use case that exercises them; the corresponding SRS Test Case IDs are listed in the traceability table in Section 13:

- UC-01 Mark Attendance: SAM-F-010 to SAM-F-013, and SAM-F-014 (modify with a reason)
- UC-02 Select Class and Subject: SAM-F-010
- UC-03 View Attendance Percentage: SAM-F-015, SAM-F-017
- UC-04 Manage User Access: SAM-F-004, SAM-F-005, SAM-F-006, SAM-F-007, SAM-SEC-002
- UC-05 Log Administrative Action: SAM-SEC-005
- UC-06 Generate Reports: SAM-F-018, SAM-F-019, SAM-F-020
- UC-07 View Attendance History: SAM-F-016, SAM-SEC-004
- Subject management (no use case of its own, tested by Anmol Vyas): SAM-F-008, SAM-F-009
- Cross-cutting: SAM-F-001, SAM-F-002, SAM-F-003 (login/authentication, exercised as a precondition throughout Section 13), SAM-SEC-001, SAM-SEC-003

4. Features Not to be Tested

- Django's own authentication internals (session handling, password hashing algorithm correctness) — the project relies on Django's built-in, well-tested implementation rather than re-testing the framework itself.
- MySQL engine-level correctness (transactions, ACID properties) — assumed reliable; the team tests its own schema constraints (e.g. the duplicate-attendance unique constraint, SAM-F-013) but not MySQL itself.
- Load beyond 20 simultaneous users and testing over a real network: out of scope. A 20-user load test (TC-Load-01) and a timing check of every page against the 3-second limit (TC-Perf-01) are automated.

- Cross-browser pixel-level UI testing — basic checks on Chrome only for this phase, consistent with SAM-NF-003; full cross-browser testing is a stretch goal, not a Phase-3 commitment.

5. Test Approach / Strategy

Levels

- Unit tests (module-level) — one module's logic in isolation, e.g. the attendance-percentage calculation formula (TC-Calc-01).
- Integration tests — two or more modules working together, e.g. the Attendance Module checking the Student Module's enrolment list before allowing a mark.
- System tests (end-to-end) — a realistic full flow a real user would perform, login through to the final result.

Types

- Functional testing — the 20 functional requirements (SAM-F-001 to SAM-F-020), traced in Section 13.
- Security testing — the 5 security requirements (SAM-SEC-001 to SAM-SEC-005), covered wherever a use case touches role-based access or data isolation (see Section 5.1).
- Regression testing — the whole automated suite runs on every pull request and on every merge to main in CI, so every change is regression-tested before it is merged.
- Performance and load testing — TC-Perf-01 times every main page against SAM-NF-001's 3 seconds with a term's worth of data; TC-Load-01 runs 20 users at once (SAM-NF-007).

Entry criteria: the relevant module has a working build deployed to the test environment; test data (sample students, faculty, subjects, classes) has been seeded; the SRS requirement being tested has not changed since this test plan was written.

Exit criteria: all High-priority functional requirement test cases pass; all five SAM-SEC test cases pass with no exceptions (security requirements are treated as non-negotiable per the SRS's acceptance criteria, Section 6.2); any Medium-priority failures are logged as known issues with a plan to fix before final submission.

5.1 Security Validation

- Verify role-based access cannot be bypassed by direct URL/request manipulation (SAM-SEC-002, TC-SEC-02).
- Verify a student cannot access another student's data by altering a student ID in a request (SAM-SEC-004, TC-SEC-04).
- Verify passwords are never stored or transmitted in plain text (SAM-SEC-003, TC-SEC-03) — to be checked once the authentication module is built, via direct database inspection.
- Verify administrative actions are recorded in an audit trail that cannot be edited or deleted through the normal UI (SAM-SEC-005, TC-SEC-05) — see UC-05 in Section 13.
- No formal penetration testing or fuzzing is planned for this phase, unlike the ATM sample reference; this is flagged as a gap to revisit if time permits before final submission.

6. Test Environment

Hardware

Standard development laptops (team members' own machines); no specialised hardware required, matching SRS Section 3.2 (Hardware Interfaces).

Software

Category	Tool / Setup
Version control	Git + GitHub (team repository)
Backend framework under test	Python 3, Django (per SRS Section 2.1)
Database	MySQL 8.0
Unit/Integration testing	pytest with pytest-django; coverage.py with branch coverage, at least 85% (CI gate)
Manual/System testing	Chrome walkthroughs by team members, plus automated end-to-end page checks through the real login page (use-case and TC-UX-01 test cases)
Bug tracking	GitHub Issues
CI	GitHub Actions on every pull request and push to main: lint (ruff), Django checks, migration check, tests on MySQL 8.0 with coverage, Bandit, SonarCloud, Docker image build
Containers	Docker and docker-compose: Nginx (HTTPS), Django with gunicorn, MySQL 8.0
Static analysis and security	SonarCloud, Bandit, ruff (lint and complexity)

Test Data

Three data sets, the same on every machine (docs/testing/test-data.md): seed_data, the sample logins, classes, subjects and time slots; tests/testdata.py, ten teaching days with a known percentage for each student; and seed_load, about 65,000 attendance rows for the performance and load tests.

Note (version 1.1): the tooling above is the working pipeline used throughout Sprints 1 and 2.

7. Test Schedule

The milestones were provisional in version 1.0. Version 1.1 records the actual dates from Sprints 1 and 2.

Milestone	Target Date
Test plan finalised (this document)	17-09-2026
Test environment / CI setup	19-10-2026 (Sprint 1)
Unit test execution begins (per module, as built)	19-10-2026, with each feature's pull request
Integration test execution	21-10-2026 to 29-10-2026
System test execution	29-10-2026 (Sprint 2: use-case and non-functional test cases)
Final test summary report	29-10-2026 (docs/testing/test-summary.md)

8. Test Deliverables

- This Test Plan document (version 1.1)
- Test cases: automated pytest test cases in the repository's tests/ folder, listed in Appendix A
- Test data / fixtures (Section 6)

- Test execution logs: every CI run on GitHub Actions, with its coverage report
- Defect reports (GitHub Issues, label 'bug')
- Test summary report: docs/testing/test-summary.md

9. Roles and Responsibilities

Roles follow the department STP template's QA Lead, Test Engineer, and Developer titles. Product Owner is not used: the Course Coordinator's sign-off in Section 15 already covers approval and readiness, and a class project has no separate product stakeholder for that role to represent. Developer is held by two members.

Role	Name	Responsibility
QA Lead	Anupriya	Documented the test environment and metrics (Sections 7-9); UC-04 and UC-05 test coverage, incl. SAM-SEC-005 checks; coordinates the plan going forward.
Test Engineer	Anusha G	Designed and validated test-case traceability (Sections 4-6); UC-03 and UC-07 test coverage, incl. SAM-SEC-004 checks.
Developer	Anmol Vyas	Restructured the document to the department's STP template (Sections 10-12); UC-06 test coverage; cross-checked Section 13 against the SRS's RTM; will support defect fixes once implementation begins.
Developer	Anupam Rajesh Tiwari	Prepared the initial document, Sections 1-3, and Section 14; UC-01 and UC-02 test coverage; will support defect fixes once implementation begins.

All four members reviewed the full document together before submission, cross-checking test data and confirming the security-related entries in Section 13 match the SRS.

10. Risks and Mitigation

Risk	Mitigation
Phase-2 Project Plan not yet complete, so schedule (Section 7) is provisional	Revisit and update Section 7 once the Phase-2 Gantt chart exists
No implementation exists yet, so no test case has been executed	Prioritise High-priority SAM-F and all SAM-SEC test cases first, once modules become available
Team members' personal laptops used as test environment, no shared server yet	Keep test data setup steps documented in Section 6 so any member can reproduce the environment
Design gaps surfaced while validating traceability (Section 13.1) could block execution if unresolved	Escalate to team discussion / course coordinator before Phase-4 implementation begins
Uneven familiarity with Django's TestCase framework across team members	Anmol and Anupam to share a short internal walkthrough before automated test writing starts

11. Assumptions and Dependencies

- The Phase-1 SRS (v1.0) requirements will not change materially before implementation begins; if they do, the traceability in Section 13 will need revision.
- Test data (sample students, faculty, subjects) will be created by the team before execution, as no institutional data feed is available for a class project.
- Django and MySQL, as fixed in SRS Section 2.1 and 2.5, remain the implementation stack; no test cases assume an alternate backend.
- Team members will have access to a working local development environment before Phase-4 begins.

12. Suspension and Resumption Criteria

Suspend testing if:

- The test environment (Section 6) is unavailable for an extended period (e.g. a team member's local setup breaks and no shared environment exists yet).
- A build is too unstable to execute a meaningful share of planned test cases for a given use case.
- A design gap noted in Section 13.1 blocks a specific use case's tests entirely (e.g. the attendance schema's session/slot gap for UC-01).

Resume testing if:

- The blocking issue is resolved (environment restored, build stabilised, or design decision made).
- The affected test cases have been reviewed and updated if the underlying requirement changed.

13. Test Case Management and Traceability

The SRS already carries a full Requirements Traceability Matrix (SRS Section 8) mapping every one of the 32 requirements — 20 functional, 7 non-functional, and 5 security — to a specific Test Case ID, owning module, and status. Repeating that matrix here would just duplicate it, so, matching how the department's own STP template treats this section (a short example table, not a full re-statement), this section summarises traceability at the use-case level and points back to the SRS for the full detail.

Use Case	SRS Requirement(s)	SRS Test Case ID(s)	Test Level(s)	Owner
UC-01: Mark Attendance	SAM-F-010 to SAM-F-014	TC-Att-01 to TC-Att-05	Unit, Integration	Anupam Rajesh Tiwari
UC-02: Select Class and Subject	SAM-F-010	TC-Att-01	Unit	Anupam Rajesh Tiwari
UC-03: View Attendance Percentage	SAM-F-015, SAM-F-017	TC-Calc-01, TC-Alert-01	Unit, Integration	Anusha G
UC-04: Manage User Access	SAM-F-004 to SAM-F-007, SAM-SEC-002	TC-Student-01 to 03, TC-Faculty-01, TC-SEC-02	Unit, Security	Anupriya
UC-05: Log Administrative Action	SAM-SEC-005	TC-SEC-05	Integration, Security	Anupriya
UC-06: Generate Reports	SAM-F-018 to SAM-F-020	TC-Report-01 to 03	Unit, System	Anmol Vyas
UC-07: View Attendance History	SAM-F-016, SAM-SEC-004	TC-View-01, TC-SEC-04	Unit, Security	Anusha G
Cross-cutting (login precondition)	SAM-F-001 to SAM-F-003	TC-Auth-01 to 03	Unit	All (shared precondition)
Subject management	SAM-F-008, SAM-F-009	TC-Subject-01, TC-Subject-02	Unit, Integration	Anmol Vyas
Non-functional	SAM-NF-001 to SAM-NF-007	TC-Perf-01, TC-Reliability-01, TC-UX-01, TC-Sec-01, TC-Recovery-01, TC-Maint-01, TC-Load-01	System	Owners in Appendix A

Version 1.1: every Test Case ID above is automated as pytest test cases in the repository (tests/). They log in through the real login page where a user would, run on every pull request in CI, and all pass. Appendix A lists each test case with its requirement, test files, owner and status.

13.1 Open Items Raised While Validating Traceability

Checking this traceability against the SRS surfaced a few gaps the SRS does not resolve on its own. The team is listing them here rather than quietly picking an answer, since these affect the design phase:

- Same-subject, multiple-sessions-per-day: the attendance schema behind SAM-F-012 (student, subject, date) cannot distinguish a lecture session from a lab session on the same day for the same subject (relevant to TC-Att-03). Needs a session/slot field added to the design before implementation, or a team decision that this is out of scope.

- Student record deletion: SRS Section 4.2 (SAM-F-004 to SAM-F-006) specifies add/update/search but not delete. The team proposes deactivation instead of hard delete, to avoid orphaning historical attendance records, but this should be confirmed with the course coordinator or treated as a team design decision documented in Phase-2.
- Audit logging of denied/failed actions: SAM-SEC-005 (TC-SEC-05) says "important attendance modifications and administrative actions" should be logged — it is not explicit whether a denied unauthorized attempt counts. The team leans towards logging these too since they are security-relevant, but is flagging this for team agreement.
- Report generation performance: SAM-NF-001 (TC-Perf-01) cannot be measured until a build exists to time against a realistically sized dataset; this is a placeholder to run once implementation exists.
- Default sort order for attendance history: SAM-F-016 (TC-View-01) does not specify newest-first vs oldest-first; the team has suggested newest-first as more useful but this is a minor UI decision, not a blocker.

Version 1.1: all five were resolved in SAD Section 4.6 and implemented in Sprints 1 and 2: one-hour time slots (ADR-004, SRS v1.1), deactivation instead of deletion (ADR-003), DENIED and FAILED actions logged (ADR-005), report performance measured by TC-Perf-01, and history shown newest first.

14. Test Metrics and Reporting

Metrics collected

- % of test cases executed, against the 32 requirements tracked in the SRS's RTM (SRS Section 8)
- % passed / failed, broken down by use case
- Defect density (defects per module, e.g. Attendance Module vs Report Module)
- Requirement coverage — every SAM-F, SAM-NF, and SAM-SEC ID mapped to at least one executed test case, verifying that the SRS's RTM holds up under actual execution

Reporting

The Sprint 2 test summary report (docs/testing/test-summary.md) gives the results: 358 automated tests passing, 32 of 32 requirements covered, all seven use cases tested, 96% line and branch coverage, and no open defects. Progress was tracked on the GitHub Project board (SAMS Backlog).

15. Approvals

Role	Name	Signature / Date
QA Lead	Anupriya	Approved — 17-09-2026
Test Engineer	Anusha G	Approved — 17-09-2026
Developer	Anmol Vyas	Approved — 17-09-2026
Developer	Anupam Rajesh Tiwari	Approved — 17-09-2026
Course Coordinator	Asha Kumari (asha.hebbar@gmail.com)	Pending

Conclusion

This test plan sets the testing strategy, environment, schedule, and responsibilities for the Student Attendance Management System's seven use cases, across Unit, Integration, System, and dedicated Security testing levels, directly traceable to the 20 functional, 7 non-functional, and 5 security requirements specified in the Phase-1 SRS. It follows the department's 15-section Software Test Plan template structure, including that template's lightweight treatment of Section 13: rather than duplicating the SRS's own Requirements Traceability Matrix (Section 8), Section 13 here summarises it at the use-case level and points back to the SRS for full detail.

Five open design questions surfaced while validating that traceability are tracked in Section 13.1 and will be carried forward into the Phase-2 planning and design discussions, rather than being silently resolved by whoever happened to notice them. Version 1.1 records how the plan was carried out in Sprints 1 and 2.

Team 3: Anmol Vyas, Anupam Rajesh Tiwari, Anupriya, Anusha G.